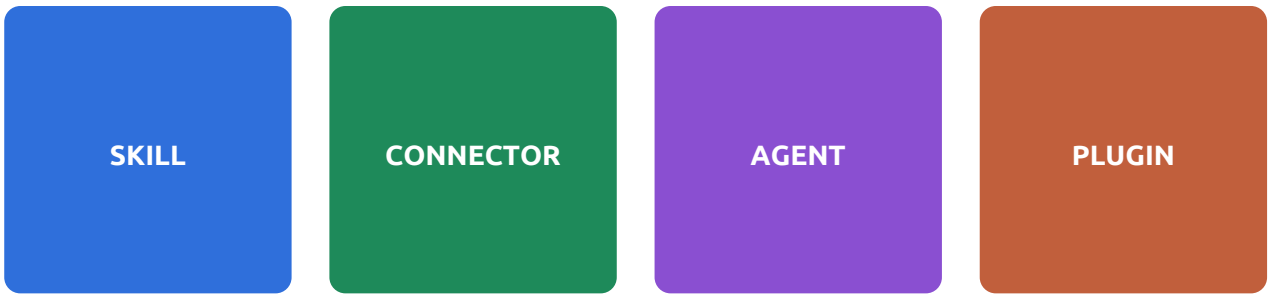
# Skills, Connectors, Plugins and Agents in Claude Code

What each one is, how to define it in the Claude CLI and the VS Code extension, how to use it day to day, and how to package all of it in an npm package so several projects can share one copy.




Covers Claude Code 2.1.x (CLI and VS Code extension) · October 2026
File and field names were checked against the official docs at code.claude.com/docs.

# Contents

1. The short version 3

How they relate . . . 4

Context cost: why the distinction matters . . . 4

2. Where configuration lives (CLI and VS Code) 5

3. Skills 6

3.1 Define a skill . . . 6

3.2 Where to put it (scope) . . . 7

3.3 Use a skill . . . 8

4. Connectors (MCP servers) 8

4.1 Define a connector . . . 8

4.2 Authenticate and use . . . 9

5. Agents (subagents) 10

5.1 Define an agent . . . 10

5.2 Use an agent . . . 11

6. Plugins 12

6.1 Define a plugin . . . 12

6.2 Develop and test locally . . . 13

6.3 Publish through a marketplace . . . 13

6.4 Install and use . . . 13

7. Which one should I use? 14

8. Packaging in npm and reusing across projects 15

8.1 Route A: a plugin as an npm package (recommended) . . . 15

8.2 Route B: a connector as an npm package . . . 16

8.3 Route D: copying files (only if you must) . . . 17

8.4 Route C: reuse from code with the Claude Agent SDK . . . 17

9. End-to-end example: one kit, many repositories 19

10. Cheat sheet 20

# 1. The short version

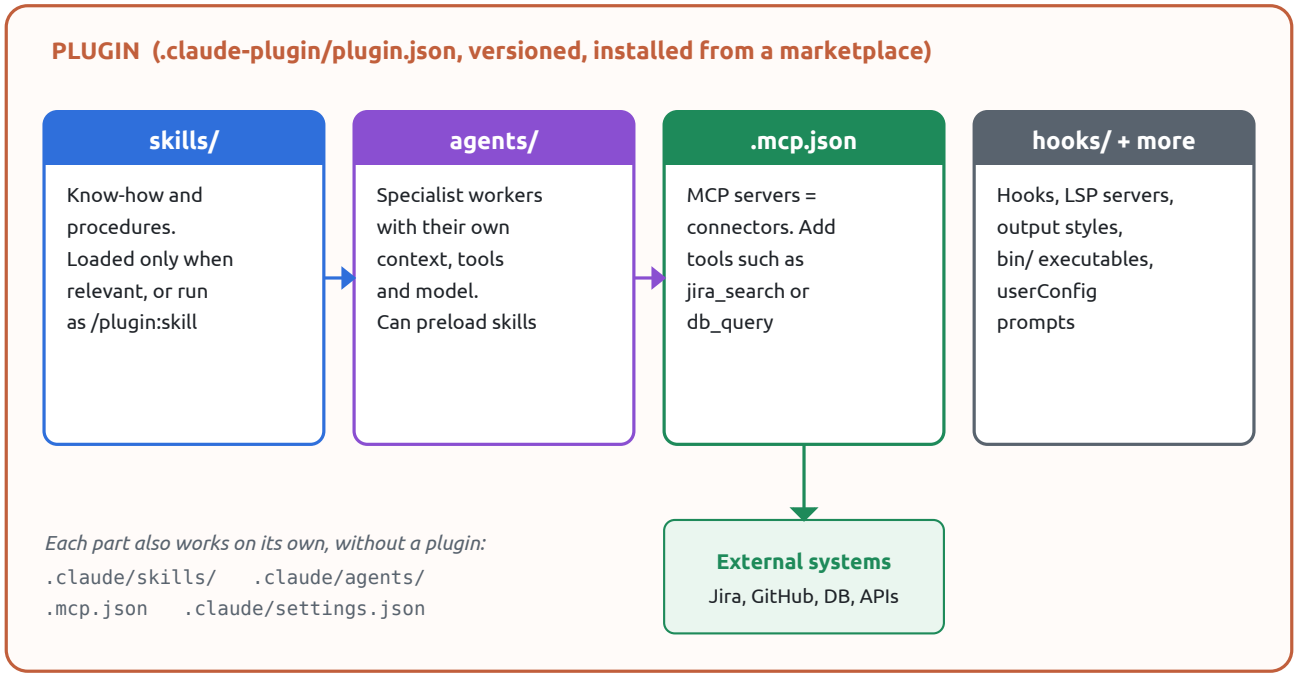
These four ideas are easy to mix up because they overlap: a plugin can contain the other three, and a skill can start an agent. Each one answers a different question:

| Concept | It answers | What it really is | Lives in |
|---|---|---|---|
| **Skill** | "*How* should Claude do this task?" | A folder with a `SKILL.md` file: instructions, plus optional scripts and reference files. Claude loads it when the task matches its description, or you run it as `/name`. | `.claude/skills/<name>/SKILL.md` |
| **Connector (MCP server)** | "*What outside system* can Claude reach?" | A Model Context Protocol server that gives Claude new **tools** (and resources), such as Jira, GitHub, Postgres or Gmail. "Connector" is the claude.ai name; Claude Code calls them MCP servers. | `.mcp.json`, `~/.claude.json`, or claude.ai settings |
| **Agent (subagent)** | "*Who* should do this work?" | A specialist Claude with its own system prompt, tool allowlist, model and **separate context window**. The main Claude hands it a task and gets back a summary. | `.claude/agents/<name>.md` |
| **Plugin** | "How do I *ship* all of this together?" | A distributable bundle that can hold skills, agents, hooks, MCP servers, LSP servers, output styles and more, with a manifest and a version. Installed from a **marketplace**. | `.claude-plugin/plugin.json` + folders |

**An analogy.** Think of Claude as a new engineer on your team:

- A **skill** is a runbook or playbook on the team wiki ("how we cut a release").
- A **connector** is an account on a system (Jira login, database credentials).
- An **agent** is a colleague you hand a task to ("ask the security reviewer to look at this").
- A **plugin** is the onboarding kit that installs the playbooks, accounts and colleague contacts in one go.

## How they relate




Two points that clear up most of the confusion:

- **Skills and agents are both Markdown files with YAML frontmatter**, but they behave differently. A skill's text is *added to the current conversation*. An agent's text becomes *the system prompt of a separate Claude* that runs in its own context window and only returns a result.
- **Skills and agents only change what Claude knows and does. Connectors give Claude new abilities.** A skill can say "file a Jira ticket", but Claude can only do it if a Jira connector (MCP server) is installed, or some other route exists, such as a CLI the skill tells it to run.

## Context cost: why the distinction matters

| | What sits in context all the time | What loads on demand |
|---|---|---|
| **Skill** | Only the name and `description` (a short line per skill) | The full `SKILL.md` body when invoked; extra files only if Claude opens them |
| **Connector** | Tool names and schemas (Claude Code defers large tool sets behind a tool search) | Tool results, when a tool is called |
| **Agent** | Only the name and `description` | Nothing in *your* context: the agent works in its own window and returns a summary |
| **Plugin** | Whatever its parts add, as above | Same as its parts |

This progressive loading is why you can install dozens of skills without filling the context window, and why agents are the right tool for noisy work such as searching a large codebase or reading long logs.

# 2. Where configuration lives (CLI and VS Code)

**The CLI and the VS Code extension use the same files.** The extension bundles its own copy of the Claude Code CLI and reads the same `~/.claude/` and `<project>/.claude/` folders, so anything you define once works in both. The docs say: "Plugins and marketplaces you configure in the extension are also available in the CLI, and vice versa."

**Folder map**

```
~/.claude/                      # USER scope: applies to every project on this machine
  settings.json                 #   permissions, env, hooks, enabledPlugins, marketplaces
  skills/<name>/SKILL.md        #   personal skills
  agents/<name>.md              #   personal subagents
  commands/<name>.md            #   legacy custom commands (still work; now part of skills)
  plugins/                      #   plugin cache (managed by Claude Code; don't edit)
~/.claude.json                  # MCP servers at user + local scope (via `claude mcp`)

<project>/                      # PROJECT scope: commit to git so the team shares it
  CLAUDE.md                     #   always-on project instructions (memory)
  .mcp.json                     #   project MCP servers (connectors)
  .claude/
    settings.json               #   shared: enabledPlugins, marketplaces, hooks
    settings.local.json         #   your personal overrides (git-ignored)
    skills/<name>/SKILL.md      #   project skills
    agents/<name>.md            #   project subagents
```

| Task | Claude CLI (terminal) | VS Code extension (chat panel) |
|---|---|---|
| **Run a skill** | Type `/skill-name args` | Type `/skill-name args` in the prompt box; `/skills` opens a list with visibility toggles |
| **Manage connectors** | `claude mcp add \| list \| get \| remove`; `/mcp` in a session | `/mcp` opens a dialog (add, remove, authenticate), or run `claude mcp add` in the integrated terminal |
| **Use an agent** | Ask for it, `@agent-name`, or `claude --agent name` | Ask for it or @-mention it. The **agent map** (the "2 agents" counter) shows running subagents |
| **Manage plugins** | `/plugin` panel or `claude plugin install \| list \| validate` | `/plugins` opens **Manage plugins** (Plugins and Marketplaces tabs) |
| **Settings** | Edit `settings.json` or use `/config` | Same files; the Customize section links to them |

**VS Code tip.** Installing the VS Code extension does **not** put `claude` on your shell PATH. To run `claude mcp add` or `claude plugin ...` in a terminal, install the standalone CLI once. After that, both tools read and write the same configuration.

# 3. Skills

A **skill** is a folder that teaches Claude how to do one kind of task. It holds a required `SKILL.md` and any supporting files (templates, scripts, reference docs). Claude sees the skill's name and description in every session. When your request matches the description, Claude loads the full instructions. You can also run it yourself as a slash command.

Custom slash commands (`.claude/commands/deploy.md`) have been merged into skills: that file and `.claude/skills/deploy/SKILL.md` both create `/deploy`. Old command files keep working, but new work should use skills, which add supporting files, automatic loading and invocation control.

## 3.1 Define a skill

**Layout**

```
.claude/skills/release-notes/
  SKILL.md              # required: frontmatter + instructions
  template.md           # optional: Claude reads it when the instructions say so
  scripts/
    collect_prs.sh      # optional: Claude runs it with the Bash tool
```

**.claude/skills/release-notes/SKILL.md**

```
---
name: release-notes
description: Drafts release notes from merged PRs since the last git tag. Use when the
  user asks for release notes, a changelog entry, or "what shipped".
argument-hint: "[version]"
allowed-tools: Bash(git log:*) Bash(gh pr list:*) Read
---

# Release notes

## Context (runs before Claude reads this skill)
Last tag: !`git describe --tags --abbrev=0`

## Steps
1. Run `${CLAUDE_SKILL_DIR}/scripts/collect_prs.sh` to list PRs merged since the last tag.
2. Group them into Features, Fixes and Internal, using template.md as the format.
3. Title the notes "Release $ARGUMENTS". If no version was given, ask for one.
4. Leave out PRs labelled `skip-changelog`.
```

### Frontmatter fields you will use most

| Field | Purpose |
|---|---|
| `name` | Command name in the `/` menu. Defaults to the folder name. |
| `description` | **The most important field.** What the skill does and when to use it; Claude decides whether to load the skill from this text alone. Put the key use case first and include trigger phrases. |
| `when_to_use` | Extra trigger context, appended to the description. |
| `argument-hint` | Autocomplete hint such as `[issue-number]`. |
| `disable-model-invocation` | `true`: only you can run it (with `/name`). Use this for skills with side effects, such as deploy. |
| `user-invocable` | `false`: hidden from the `/` menu; only Claude invokes it (background know-how). |
| `allowed-tools` | Tools Claude may use without asking while the skill runs, for example `Read Grep Bash(npm test:*)`. |
| `model` / `effort` | Model or reasoning effort for the rest of the turn. |
| `context: fork` + `agent` | Run the skill in a forked subagent (for example `agent: Explore`) so its work stays out of the main context. |
| `paths` | Glob patterns that limit automatic loading to matching files. |
| `hooks` | Hooks that are active only while the skill runs. |

### Placeholders inside the body

| Placeholder | Becomes |
|---|---|
| `$ARGUMENTS` | Everything typed after the command, e.g. `/release-notes 2.4.0` gives `2.4.0` |
| `$0`, `$1` ... (or `$ARGUMENTS[N]`) | One argument by position (0-based). Quote multi-word values. |
| `$name` | A named argument declared in the `arguments:` frontmatter |
| `${CLAUDE_SKILL_DIR}` | Absolute path of the skill's folder, for calling bundled scripts |
| !`command` | Runs a shell command *before* Claude reads the skill and inserts its output (dynamic context). Use a ```! fenced block for several lines. If a command fails, the skill does not run. |

## 3.2 Where to put it (scope)

| Scope | Path | Who gets it |
|---|---|---|
| **Personal** | `~/.claude/skills/<name>/SKILL.md` | You, in every project |
| **Project** | `.claude/skills/<name>/SKILL.md` | Everyone who clones the repo |
| **Nested** | `<subdir>/.claude/skills/<name>/SKILL.md` | Sessions started in or below that folder (monorepos) |
| **Plugin** | `<plugin>/skills/<name>/SKILL.md` | Anyone with the plugin enabled; runs as `/plugin:skill` |
| **Enterprise** | Managed settings directory | The whole organization |

If two skills share a name, enterprise wins over personal, and personal wins over project. A skill also wins over a `.claude/commands/` file of the same name.

### 3.3 Use a skill

- **Automatically:** just ask, for example "write the release notes for 2.4.0". Claude matches the description and loads the skill. This is the main way skills are meant to work.
- **Explicitly:** type `/release-notes 2.4.0` in the CLI or in the VS Code prompt box. Plugin skills are namespaced: `/my-plugin:release-notes`.
- **From an agent:** list the skill in a subagent's `skills:` field to preload it when the agent starts.
- **Check what is loaded:** type `/` to browse, or `/skills` in VS Code to see each skill's visibility.

> **Portable.** Skills follow the open **Agent Skills** format, so the same folder also works in claude.ai, Claude Desktop and the Claude Agent SDK. Write each skill once and reuse it on every surface.

## 4. Connectors (MCP servers)

A **connector** links Claude to an outside system through the **Model Context Protocol (MCP)**. An MCP server exposes **tools** (actions such as `create_issue`), **resources** (readable data, referenced with `@server:resource`) and **prompts** (which show up as slash commands). The names differ by product, but the mechanism is the same:

- **claude.ai / Claude Desktop** call them **Connectors**. You add them at *claude.ai > Settings > Connectors*, usually remote servers with OAuth sign-in.
- **Claude Code** calls them **MCP servers**. You add them with `claude mcp add` or in `.mcp.json`. When you sign in to Claude Code with a claude.ai subscription, the connectors from your claude.ai account **also appear in Claude Code**, with no extra setup.

### 4.1 Define a connector

#### Option A: the CLI (works in the VS Code terminal too)

Terminal

```
# Remote server over HTTP (recommended for hosted services)
claude mcp add --transport http notion https://mcp.notion.com/mcp

# Remote server with a static auth header
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer $API_TOKEN"

# Local process over stdio (often an npm package run with npx)
claude mcp add --transport stdio --env AIRTABLE_API_KEY=YOUR_KEY airtable \
  -- npx -y airtable-mcp-server
#      ^^ everything after `--` is the command that starts the server

# Choose where it is saved
claude mcp add --scope project --transport http sentry https://mcp.sentry.dev/mcp

claude mcp list            # show all servers and their status
claude mcp get sentry      # details for one server
claude mcp remove sentry
```

| Scope (--scope) | Stored in | Shared? | Use for |
|---|---|---|---|
| `local` **(default)** | `~/.claude.json`, under this project's path | No | Personal or experimental servers, and secrets |
| `project` | `.mcp.json` in the repo root | Yes, through git | Team tools that every developer should have |
| `user` | `~/.claude.json` | No | Your own tools, in every project |

### Option B: commit a .mcp.json (project scope)

**.mcp.json**

```
{
  "mcpServers": {
    "jira": {
      "type": "http",
      "url": "${JIRA_MCP_URL:-https://mcp.atlassian.com/v1/mcp}"
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@acme/postgres-mcp"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    }
  }
}
```

`${VAR}` and `${VAR:-default}` are expanded in `command`, `args`, `env`, `url` and `headers`, so secrets stay in each developer's environment and out of git. Claude Code asks each person to approve project-scoped servers the first time it sees them (`claude mcp reset-project-choices` resets those answers).

### Option C: claude.ai connectors

Add the connector once on claude.ai. It then appears in Claude Code when you are signed in with that subscription. It does **not** load when you use an API key, `apiKeyHelper`, or a provider such as Amazon Bedrock. Turn it off per connector in `/mcp`, or turn all of them off with `"disableClaudeAiConnectors": true` in settings.

## 4.2 Authenticate and use

- **OAuth servers:** run `/mcp` in a session (CLI or VS Code), choose the server and finish sign-in in the browser. Tokens are stored securely and refreshed for you. From a shell: `claude mcp login <name>`.
- **Using the tools:** just ask ("find open P1 bugs in Jira assigned to me"). Claude picks the tool. Tools are named `mcp__<server>__<tool>`, for example `mcp__jira__search`, and plugin servers are named `mcp__plugin_<plugin>_<server>__<tool>`.
- **Permissions:** use those names in `settings.json` allow and deny rules, in a skill's `allowed-tools`, or in an agent's `tools`, for example `"allow": ["mcp__jira__search"]`.
- **Resources:** type `@` to reference server resources. **MCP prompts** appear as `/mcp__server__prompt` commands.

**Precedence.** If there are several servers with the same name, the order of precedence is local, then project, then user, then plugin-provided servers, then claude.ai connectors. The whole entry from the winning source is used; fields are not merged.

# 5. Agents (subagents)

A **subagent** is a specialist instance of Claude that the main session can hand work to. Each subagent has:

- its own **context window**: it reads files and runs commands without filling yours, and sends back a summary;
- its own **system prompt**: the Markdown body of its file;
- its own **tool allowlist** and **model**, for example a read-only reviewer on a faster model;
- optional preloaded **skills**, scoped **MCP servers**, **hooks**, persistent **memory** and git **worktree isolation**.

Claude Code includes built-in agents such as **Explore** (fast, read-only codebase search), **Plan** and **general-purpose**. You add your own.

## 5.1 Define an agent

**.claude/agents/security-reviewer.md**

```
---
name: security-reviewer
description: Reviews code changes for security issues (injection, authz, secrets,
  unsafe deserialization). Use proactively after edits to auth, payments or API code.
tools: Read, Grep, Glob, Bash(git diff:*)
model: sonnet
skills: owasp-checklist
color: red
---

You are a senior application-security reviewer.

When invoked:
1. Run `git diff HEAD` to see what changed.
2. Check each change against the owasp-checklist skill.
3. Report findings as: severity, file:line, issue, concrete fix.
Do not edit files. If nothing is wrong, say so in one line.
```

| Field | Meaning |
|---|---|
| `name` / `description` | Required. Claude decides when to delegate from the description; adding "use proactively" makes it delegate more readily. |
| `tools` / `disallowedTools` | Allowlist or denylist. If you leave out `tools`, the agent inherits all tools, including MCP tools. |
| `model` | `sonnet`, `opus`, `haiku`, `fable`, a full model ID, or `inherit` |
| `skills` | Skills whose full text is preloaded into the agent at startup |
| `mcpServers` | MCP servers only this agent can use (by name, or defined inline) |
| `permissionMode` | `default`, `acceptEdits`, `plan`, `bypassPermissions` ... |
| `memory` | Persistent memory scope: `user`, `project` or `local` |
| `isolation: worktree` | Run in a temporary git worktree so its edits can't collide with yours |
| `maxTurns`, `effort`, `background`, `color` | Turn limit, reasoning effort, background running, and UI colour |

**Where agent files go** (if names clash, the higher row wins): managed settings > the `--agents` CLI flag > `.claude/agents/` (project) > `~/.claude/agents/` (user) > a plugin's `agents/` folder. Claude Code scans these folders recursively.

### Session-only agents from the command line

**Terminal (not saved to disk)**

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after code changes.",
    "prompt": "You are a senior code reviewer. Focus on quality and security.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  }
}'
```

## 5.2 Use an agent

- **Automatic delegation:** Claude chooses the agent when the task fits its description.
- **By name:** "Use the security-reviewer agent to review my last commit."
- **@-mention (guaranteed):** `@agent-security-reviewer check the auth changes`. Plugin agents use the namespaced form `@agent-my-plugin:security-reviewer`.
- **As the whole session:** `claude --agent security-reviewer`, or set `"agent": "security-reviewer"` in `.claude/settings.json` to make it the default.
- **In VS Code:** the same prompts and @-mentions work in the chat panel. Running subagents appear as live rows and in the **agent map** (click the "N agents" counter, or type `/tasks`), where you can read their transcript, stop them, or move them to the background.
- **Managing them:** in current versions, edit the files in `.claude/agents/` directly, or ask Claude to write one ("create a subagent that ...").

**Two meanings of "agent".** "Agent" can also mean an agent you *build* with the **Claude Agent SDK** (`@anthropic-ai/claude-agent-sdk` for TypeScript, `claude-agent-sdk` for Python). That is the same engine as Claude Code, used as a library in your own app. It accepts the same subagent definitions (the `agents` option), MCP servers and plugins. See section 8.4.

# 6. Plugins

A **plugin** is the packaging and distribution unit. It bundles any mix of skills, agents, hooks, MCP servers (connectors), LSP servers, output styles and helper executables under one name and version. People install it from a **marketplace**, which is a catalog file usually hosted in a git repository. Every component in a plugin is namespaced by the plugin name, so `hello` in plugin `acme-dev-kit` runs as `/acme-dev-kit:hello`.

## 6.1 Define a plugin

**Standard layout. Only plugin.json goes inside .claude-plugin/; everything else is at the root**

```
acme-dev-kit/
  .claude-plugin/
    plugin.json            # manifest (optional; name is the only required field)
  skills/
    release-notes/SKILL.md
    owasp-checklist/SKILL.md
  agents/
    security-reviewer.md
  hooks/
    hooks.json             # e.g. run the formatter after every Edit/Write
  .mcp.json                # connectors that ship with the plugin
  bin/                     # executables put on the Bash tool's PATH while enabled
  scripts/
    format.sh
```

**.claude-plugin/plugin.json**

```
{
  "name": "acme-dev-kit",
  "displayName": "ACME Dev Kit",
  "version": "1.3.0",
  "description": "ACME release, review and Jira tooling for Claude Code",
  "author": { "name": "ACME Platform Team", "email": "platform@acme.dev" },
  "license": "MIT",
  "keywords": ["release", "security", "jira"],
  "userConfig": {
    "jira_token": {
      "type": "string", "title": "Jira API token",
      "description": "Personal token used by the Jira MCP server", "sensitive": true
    }
  }
}
```

**.mcp.json inside the plugin**

```
{
  "mcpServers": {
    "jira": {
      "command": "node",
      "args": ["${CLAUDE_PLUGIN_ROOT}/servers/jira.js"],
      "env": { "JIRA_TOKEN": "${user_config.jira_token}" }
    }
  }
}
```

Variables available to plugin components: `${CLAUDE_PLUGIN_ROOT}` (where the installed plugin lives), `${CLAUDE_PLUGIN_DATA}` (a data folder that persists across updates, for caches or `node_modules`), `${CLAUDE_PROJECT_DIR}` and `${user_config.KEY}`. Never hard-code absolute paths, because the install location changes with every version.

## 6.2 Develop and test locally

**Terminal**

```
claude --plugin-dir ./acme-dev-kit       # load it for one session, no install needed
/reload-plugins                          # inside the session, after editing files
claude plugin validate ./acme-dev-kit    # check the manifest, paths, MCP entries
claude plugin init                       # scaffold a new plugin
```

## 6.3 Publish through a marketplace

**acme-claude-marketplace/.claude-plugin/marketplace.json (push this repo to GitHub or any git host)**

```
{
  "name": "acme-tools",
  "owner": { "name": "ACME Platform Team" },
  "description": "Internal Claude Code plugins",
  "plugins": [
    { "name": "acme-dev-kit", "source": "./plugins/acme-dev-kit",
      "description": "Release, review and Jira tooling" }
  ]
}
```

## 6.4 Install and use

**CLI**

```
# CLI (shell)
# source can be a GitHub owner/repo, a git URL or a local path
claude plugin marketplace add acme/acme-claude-marketplace
claude plugin install acme-dev-kit@acme-tools      # <plugin>@<marketplace>
claude plugin list
claude plugin details acme-dev-kit

# Inside a session (CLI)
/plugin                     # interactive panel: browse, install, enable, disable
/plugin marketplace add acme/acme-claude-marketplace
/plugin install acme-dev-kit@acme-tools
```

**VS Code:** type `/plugins` in the prompt box to open **Manage plugins**. In the **Marketplaces** tab you add a source; in the **Plugins** tab you click **Install** and choose a scope (for you, or for this project). A form then asks for any `userConfig` values. You can also share a one-click link such as `vscode://anthropic.claude-code/install-plugin?plugin=acme-dev-kit&marketplace=acme/acme-claude-marketplace`.

### Roll it out to a whole team with no manual steps

**/.claude/settings.json (commit it)**

```
{
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": { "source": "github", "repo": "acme/acme-claude-marketplace" }
    }
  },
  "enabledPlugins": {
    "acme-dev-kit@acme-tools": true
  }
}
```

When a teammate opens the project, Claude Code prompts them to trust the marketplace and install the plugin. Once installed, its parts work as described in sections 3 to 5: the skills run as `/acme-dev-kit:release-notes` (or load automatically), the agents as `@agent-acme-dev-kit:security-reviewer`, the MCP tools as `mcp__plugin_acme-dev-kit_jira__*`, and the hooks fire on their events.

# 7. Which one should I use?

| If you want to... | Use | Why |
|---|---|---|
| **Teach Claude your team's way of doing a task (code style, release process, PR template)** | Skill | Cheap on context, loads only when relevant, can bundle scripts |
| **Let Claude read or write data in Jira, GitHub, a database or an internal API** | Connector (MCP) | Only MCP (or a CLI that Claude runs) gives Claude new actions |
| **Hand off a noisy or specialised job (review, research, log triage) without filling your context** | Agent | Separate context window, restricted tools, a different model if you want one |
| **Run several jobs in parallel** | Agents | Several subagents can run at once |
| **Make something happen *every time* (format after edit, block `rm -rf`)** | Hook | Hooks are deterministic; skills and agents are Claude's judgement |
| **Facts Claude needs in *every* session for this repo** | `CLAUDE.md` | Always loaded; keep it short |
| **Share any of the above with other people or projects, with versions and updates** | Plugin | One install, namespaced, versioned, can be required by team settings |

> **Combining them.** A common pattern: a **skill** describes the workflow ("to triage a bug: ..."), an **MCP server** provides the Jira and Sentry tools it needs, an **agent** does the heavy investigation in its own context, and a **plugin** packages all three so every repo gets the same setup with one line of settings.

# 8. Packaging in npm and reusing across projects

**Short answer: yes.** npm is a supported way to distribute these pieces. Which route is best depends on what you are sharing:

| Route | What it shares | How projects consume it | Recommendation |
|---|---|---|---|
| **A. Plugin published to npm, listed in a marketplace with an `npm` source** | Skills, agents, hooks, MCP servers: everything | `enabledPlugins` in `.claude/settings.json`, or `claude plugin install` | **Best for Claude Code users.** Versioned, namespaced, auto-updatable |
| **B. MCP server published to npm** | One connector | `npx -y @scope/pkg` in `.mcp.json` or `claude mcp add` | Standard practice for connectors. Works in any MCP client, not only Claude |
| **C. Library for the Agent SDK** | Agents, skills, MCP servers and plugins used from *your own code* | `import` it and pass it to `query()` | For apps, CI bots and services built on the SDK |
| **D. npm package that copies files into `.claude/`** | Raw skills and agents | A `postinstall` or `npx` script copies the files | Works, but not recommended: no namespacing, updates or uninstall |

## 8.1 Route A: a plugin as an npm package (recommended)

Claude Code can fetch a plugin straight from an npm registry, including a private one. The package *is* the plugin folder from section 6.

**Package layout**

```
@acme/claude-dev-kit/
  package.json
  .claude-plugin/plugin.json
  skills/...
  agents/...
  hooks/hooks.json
  .mcp.json
  servers/jira.js
```

**package.json**

```
{
  "name": "@acme/claude-dev-kit",
  "version": "1.3.0",
  "description": "ACME Claude Code plugin: skills, agents, Jira MCP",
  "files": [".claude-plugin", "skills", "agents", "hooks", ".mcp.json", "servers"],
  "license": "MIT",
  "publishConfig": { "registry": "https://npm.acme.dev" }
}
```

**Publish**

```
claude plugin validate .        # validate before you publish
npm publish                     # or: npm publish --access public
```

**Point a marketplace at the npm package.** npm is supported as a *plugin* source but not as a *marketplace* source, so you still need a small catalog. You have two options:

**Option 1: a git-hosted marketplace.json (one catalog for all your plugins)**

```
{
  "name": "acme-tools",
  "owner": { "name": "ACME Platform Team" },
  "plugins": [
    {
      "name": "acme-dev-kit",
      "source": {
        "source": "npm",
        "package": "@acme/claude-dev-kit",
        "version": "^1.3.0",
        "registry": "https://npm.acme.dev"
      }
    }
  ]
}
```

**Option 2: an inline catalog in each project's .claude/settings.json (no catalog repo needed)**

```
{
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": {
        "source": "settings",
        "name": "acme-tools",
        "owner": { "name": "ACME Platform Team" },
        "plugins": [
          { "name": "acme-dev-kit",
            "source": { "source": "npm",
                        "package": "@acme/claude-dev-kit", "version": "^1.3.0" } }
        ]
      }
    }
  },
  "enabledPlugins": { "acme-dev-kit@acme-tools": true }
}
```

Every project that commits that settings file gets the same plugin. To release a new version, run `npm publish`; projects pick it up within the `version` range you set.

**Important.** For safety, Claude Code fetches npm plugins with your npm client but **never runs the package's install scripts** (`preinstall`, `postinstall`). If the package ships a lockfile, dependencies are installed in a separate step, also with scripts disabled. Do not rely on `postinstall` to set anything up; use `${CLAUDE_PLUGIN_DATA}` and a `SessionStart` hook if you need a first-run step.

## 8.2 Route B: a connector as an npm package

Most MCP servers are distributed this way. Write the server with `@modelcontextprotocol/sdk`, add a `bin` entry, publish it, and every project (and every MCP client) runs it through `npx`:

**src/index.ts (simplified)**

```
#!/usr/bin/env node
// package.json: "bin": { "acme-jira-mcp": "dist/index.js" }, "type": "module"
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({ name: "acme-jira", version: "1.0.0" });
server.tool("search_issues", { jql: z.string() }, async ({ jql }) => {
  const url = `${process.env.JIRA_URL}/rest/api/3/search?jql=${encodeURIComponent(jql)}`;
  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${process.env.JIRA_TOKEN}` },
  });
  return { content: [{ type: "text", text: JSON.stringify(await res.json()) }] };
});
await server.connect(new StdioServerTransport());
```

**Use it in any project**

```
claude mcp add --scope project acme-jira --env JIRA_URL=https://acme.atlassian.net \
  -- npx -y @acme/jira-mcp@1
```

## 8.3 Route D: copying files (only if you must)

A package can ship `skills/` and `agents/` folders plus a small script, such as `npx @acme/claude-kit init`, that copies them into the project's `.claude/`. This is easy to understand, but the copies drift out of date, nothing is namespaced, and there is no clean uninstall. Use Route A instead whenever the consumers run Claude Code.

## 8.4 Route C: reuse from code with the Claude Agent SDK

If you build your own agent (a CI reviewer, an internal bot), the SDK can load the same plugin folder, often straight from `node_modules`, and accepts agents and MCP servers in code:

**ci-review.ts**

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const msg of query({
  prompt: "Review the diff on this branch and draft release notes for 2.4.0",
  options: {
    plugins: [{ type: "local", path: "./node_modules/@acme/claude-dev-kit" }],
    agents: {
      "security-reviewer": {
        description: "Reviews diffs for security issues",
        prompt: "You are a senior application-security reviewer...",
        tools: ["Read", "Grep", "Glob"],
        model: "sonnet",
      },
    },
    mcpServers: {
      jira: {
        command: "npx",
        args: ["-y", "@acme/jira-mcp@1"],
        env: { JIRA_TOKEN: process.env.JIRA_TOKEN! },
      },
    },
    settingSources: ["project"],   // also load the repo's .claude/ folder
  },
})) {
  if (msg.type === "result") console.log(msg.result);
}
```

# 9. End-to-end example: one kit, many repositories

Goal: every repository at ACME gets the same release-notes skill, security-review agent and Jira connector, and updates arrive automatically.

| Step | Action | Command or file |
|---|---|---|
| **1** | Create the plugin folder with the skill, agent and .mcp.json from sections 3 to 6 | `claude plugin init`, then edit |
| **2** | Test it locally in a real repo | `claude --plugin-dir ../acme-dev-kit` |
| **3** | Validate it | `claude plugin validate ../acme-dev-kit` |
| **4** | Publish it to the (private) npm registry | `npm publish` |
| **5** | Create a catalog that points at the npm package (git repo, or inline settings) | `marketplace.json` with an `npm` source |
| **6** | In each repo, commit `.claude/settings.json` with `extraKnownMarketplaces` + `enabledPlugins` | See 8.1, option 2 |
| **7** | Developers open the repo in the CLI or VS Code and accept the trust prompt | Nothing else to install |
| **8** | Use it | `/acme-dev-kit:release-notes 2.4.0`, `@agent-acme-dev-kit:security-reviewer` |
| **9** | Ship an update | Bump `version`, `npm publish`; users get it on the next update check |

# 10. Cheat sheet

| | Skill | Connector (MCP) | Agent | Plugin |
|---|---|---|---|---|
| **Defined in** | `SKILL.md` in a folder | `.mcp.json` / `claude mcp add` / claude.ai | `<name>.md` in an agents folder | `.claude-plugin/plugin.json` + folders |
| **Project path** | `.claude/skills/` | `.mcp.json` | `.claude/agents/` | Installed into `~/.claude/plugins/` |
| **User path** | `~/.claude/skills/` | `~/.claude.json` | `~/.claude/agents/` | Cache in `~/.claude/plugins/` |
| **Adds** | Instructions and scripts | New tools and data | A separate worker with its own context | Any of the other three, plus hooks and LSP |
| **Invoke** | Automatic, or `/name` | Automatic tool calls | Automatic, by name, `@agent-x`, `--agent` | Through its parts (`/plugin:skill`) |
| **Manage** | `/skills` (VS Code), edit files | `/mcp`, `claude mcp ...` | Edit files, `--agents` | `/plugin`, `/plugins` (VS Code), `claude plugin ...` |
| **Share** | Commit it, or ship in a plugin | Commit `.mcp.json`, npm package | Commit it, or ship in a plugin | Marketplace (git or npm source) |

### Commands at a glance

```
# Skills
/skill-name [args]                  /plugin-name:skill-name [args]

# Connectors (MCP)
claude mcp add --transport http <name> <url> [--scope local|project|user]
claude mcp add <name> -- npx -y <package>      claude mcp list | get | remove | login
/mcp                                           # status, OAuth, enable/disable (CLI and VS Code)

# Agents
@agent-<name> <task>                claude --agent <name>        claude --agents '{...}'

# Plugins
claude --plugin-dir ./my-plugin     claude plugin validate ./my-plugin
claude plugin marketplace add <owner/repo | git-url | path>
claude plugin install <plugin>@<marketplace>   claude plugin list | details | uninstall
/plugin  (CLI)    /plugins  (VS Code)          /reload-plugins
```

**Official references:** code.claude.com/docs/en/skills · /mcp · /sub-agents · /plugins-reference · /plugin-marketplaces · /plugins/marketplace-reference · /vs-code